PROJETO DE COMPILADORES · **AP2**

GROVE

A linguagem de programação em que as palavras-chave são os **cheats de GTA San Andreas**.

Arquivos `.cj`, de Carl Johnson. Python 3.10+, sem nenhuma dependência.



Gabriel Lacerda · Gabriel Mendonça · Fernando Mendonça

TODO MUNDO JÁ DIGITOU ISTO

HESOYAM

1. **Vocabulário que o público conhece**
   Ninguém precisa decorar palavra nova: o cheat já está na memória de quem jogou.
2. **Cada cheat combina com o que faz**
   `KANGAROO` pula de novo e vira o laço; `GOODBYECRUELWORLD` mata o CJ e vira o `break`.
3. **O tema não dá desconto**
   Léxico, sintático, semântico, código intermediário e geração de código: o pipeline inteiro, do mesmo jeito.

DEZ PALAVRAS RESERVADAS

HESOYAM	declara variável	TURNUPTHEHEAT	if
BRINGITON	else if	TURNDOWNTHEHEAT	else
KANGAROO	while	GOODBYECRUELWORLD	break
HELLOLADIES	print	CJPHONEHOME	lê um inteiro
FULLCLIP	true	GHOSTTOWN	false

O resto é sintaxe estilo C: `{ }` delimita bloco, `;` termina instrução, `( )` agrupa, `! && ||` são os lógicos. Três tipos: inteiro, string e booleano, sem conversão implícita.

03 · UM PROGRAMA

FATORIAL DE 5

```
# fatorial de 5
HESOYAM n = 5;
HESOYAM r = 1;
KANGAROO (n > 1) {
    r = r * n;
    n = n - 1;
}
HELLOLADIES r;
```

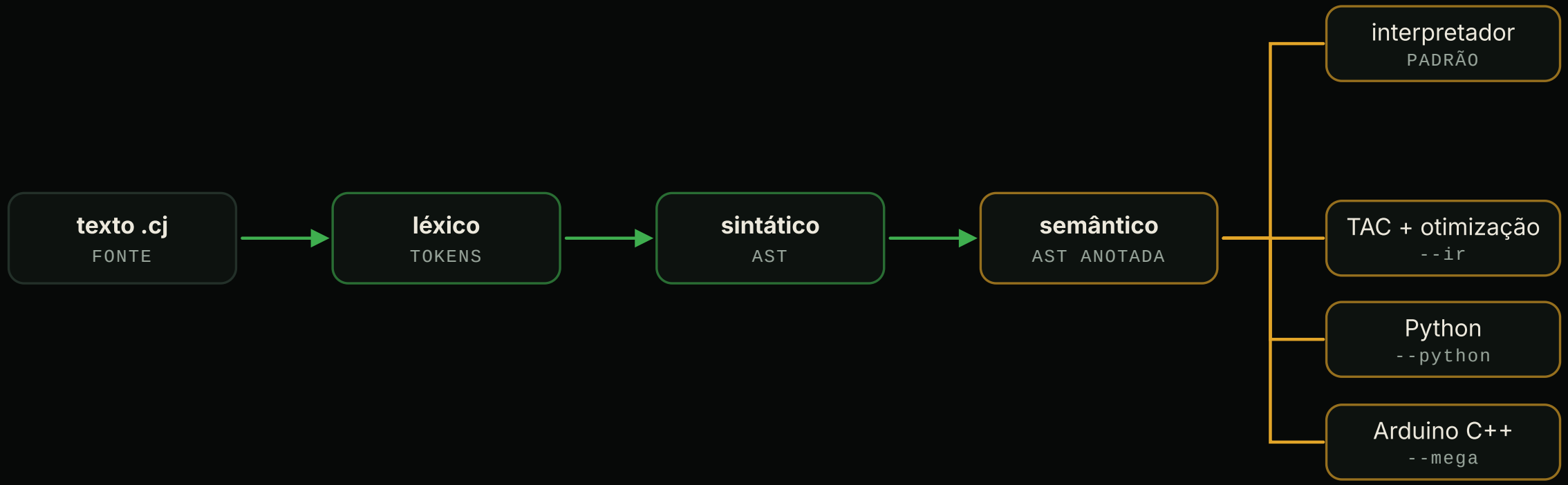
```
$ python -m grove examples/fatorial.cj
120
```

Cada execução termina como no jogo: **Mission passed!** no sucesso, Wasted no erro léxico, Busted no semântico — com o clipe de áudio do jogo, quando os `.wav` estão na pasta.

O COMPILADOR

Cinco etapas, um arquivo cada, 2.041 linhas de Python e nenhuma dependência.

DO TEXTO A QUATRO SAÍDAS



As flags param o compilador em qualquer etapa: `--tokens`, `--ast`, `--simbolos`, `--ir`. E combinam entre si.

LÉXICO: TEXTO VIRA TOKENS

```
$ python -m grove examples/fatorial.cj --tokens
LINHA:COL  TIPO     LEXEMA
2:1        VAR      HESOYAM
2:9        IDENT    n
2:11       ASSIGN   =
2:13       NUMBER   5
2:14       SEMI     ;
…
$ python -m grove examples/erro_lexico.cj
erro: linha 2, coluna 13: caractere inesperado '$'
```

- **Cinco classes**
  Cheat, pontuação e operador, número, string e identificador — duas tabelas resolvem as duas primeiras.
- **Linha e coluna em cada token**
  É o que faz o erro apontar o lugar exato, aqui e nas etapas seguintes.
- **Operador de dois caracteres primeiro**
  Senão `!=` viraria `!` seguido de `=`.

SINTÁTICO: TOKENS VIRAM AST

```
$ python -m grove examples/erro_digitacao.cj
erro: linha 2, coluna 1: 'HESOYM' não é um cheat
da Grove; você quis dizer HESOYAM?
erro: linha 4, coluna 1: 'TURNUPTHEHEET' não é um
cheat da Grove; você quis dizer TURNUPTHEHEAT?
erro: linha 5, coluna 5: 'HELLOLADYS' não é um cheat
da Grove; você quis dizer HELLOLADIES?
erro: linha 7, coluna 19: esperado expressão,
encontrado ')'
4 erros sintáticos
```

- **Descida recursiva, LL(1)**
  Um método por regra da gramática; cada regra decide olhando só o próximo token.
- **Precedência na cadeia de regras**
  `ou` → `e` → `igualdade` → `comparação` → `termo` → `fator` → `unário` → `primário`, tudo associativo à esquerda.
- **Nada de `else` ambíguo**
  Bloco é sempre `{ }`, então o pendente não existe. **`BRINGITON`** é açúcar: vira um `if` aninhado.
- **Recuperação e sugestão**
  O parser segue depois do erro e a distância de edição acha o cheat ou a variável mais parecida.

SEMÂNTICO: A AST ANOTADA

```
$ python -m grove examples/fatorial.cj --simbolos
NOME  TIPO      ESCOPO  LINHA:COL  ÚNICO  LIDA
n     inteiro   0       2:9        n      sim
r     inteiro   0       3:9        r      sim

$ python -m grove examples/erro_semantico.cj
erro: linha 4, coluna 1: 'grana' é inteiro,
não pode receber string
erro: linha 5, coluna 1: condição do TURNUPTHEHEAT
exige booleano, recebeu inteiro
erro: linha 6, coluna 17: variável 'respeito'
não declarada
erro: linha 8, coluna 1: GOODBYECRUELWORLD
fora de laço
4 erros semânticos
```

- **Tabela de símbolos**
  Pilha de escopos, um por bloco. Cada símbolo guarda tipo, escopo, posição, nome único e se foi lido.
- **Tipos sem conversão implícita**
  `1 + "a"` é erro; a condição do `if` e do `while` tem de ser booleana.
- **Todos os erros de uma vez**
  Percorre o programa inteiro antes de reclamar — não para no primeiro.
- **Anota a AST**
  Cada expressão ganha tipo e cada variável ganha seu símbolo. É dessa AST que saem a TAC, o Python e o C++.

CÓDIGO DE TRÊS ENDEREÇOS

```
$ python -m grove examples/otimizacao.cj --ir
 1   segundos = 60 * 60
 2   dobro = segundos * 2
 3   t1 = dobro + 0
 4   print t1
 5   ifFalse GHOSTTOWN goto L1
 6   print "nunca"
 7 L1:
 8 L2:
 9   ifFalse FULLCLIP goto L3
10   print "uma vez"
11   goto L3
12   goto L2
13 L3:
```

- **Uma quádrupla por instrução**
  `Quad(op, arg1, arg2, result)`: no máximo um operador, temporários `t1…tn` e rótulos `L1…Ln`.
- **Sem escopo e sem aninhamento**
  As variáveis usam o nome único da tabela de símbolos, porque a TAC é linear.
- **Curto-circuito explícito**
  `&&` e `||` viram desvios; `break` vira um `goto` para o fim do laço.
- **Executável**
  A TAC roda num executor próprio — é o que garante que ela faz o mesmo que o interpretador.

TREZE INSTRUÇÕES VIRAM QUATRO

13 instruções antes

4 depois




```
1   segundos = 3600
2   dobro = 7200
3   print 7200
4   print "uma vez"
```

- **Dobra de constantes**
  `60 * 60` é resolvido na compilação: duas contas saem do programa.
- **Propagação de cópias e constantes**
  Três valores seguem adiante sem passar por temporário.
- **Simplificação algébrica**
  `dobro + 0` é só `dobro`.
- **Desvios e código morto**
  Dois desvios resolvidos de um lado; oito instruções inalcançáveis removidas do outro.

DUAS LINGUAGENS DE SAÍDA

PYTHON · --PYTHON

```python
n = 5
r = 1
while n > 1:
    r = r * n
    n = n - 1
print(r)
```

Sai da AST, não da TAC: Python não tem `goto`, e o `if`/`while` da árvore vira `if`/`while` direto. Dá para canalizar: `--python | python3`.

ARDUINO C++ · --MEGA

```cpp
void setup() {
  Serial.begin(115200);
  _inicializar();
  long estrelas = 0L;
  _procurado(estrelas);
  while (true) {
    estrelas = _linha();   // CJPHONEHOME
    _procurado(estrelas);
    Serial.println(estrelas);
  }
}
```

Mesma AST, outro alvo: o C++ do Mega 2560, com `CJPHONEHOME` lendo a serial e o nível de procurado acendendo o HUD.

II

FORA DO TERMINAL

O mesmo compilador rodando no navegador, numa protoboard e ligado ao jogo.

GROVE ONLINE

- **O compilador em Python roda no navegador**
  Pyodide carrega o próprio pacote `grove`: nada é instalado e nenhum servidor compila nada.
- **Editor com realce dos cheats**
  Numeração, recuo automático, linha de erro marcada e o código salvo no navegador.
- **Por dentro do compilador**
  Abas com tokens, AST, tabela de símbolos, TAC, Python e o C++ do Arduino.
- **HUD do jogo como feedback**
  Uma estrela de procurado por erro encontrado, $250.000 a cada missão cumprida.

NO AR EM

grove.heybrain.uk



$250.000

Ctrl+Enter executa. Laço infinito para em 5 s. Compartilhar copia um link com o código dentro.

SEIS ESTRELAS E UM GIROFLEX

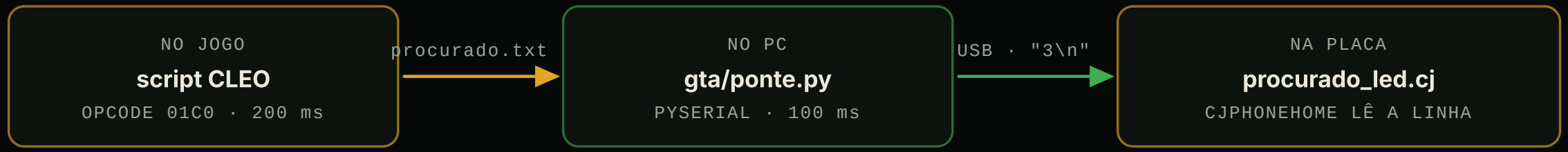


NÍVEL DE PROCURADO · 3 ESTRELAS

GIROFLEX

- **Arduino Mega 2560**
  O `.cj` compilado para C++, gravado com `arduino-cli`. Seis LEDs de estrela mais o giroflex azul e vermelho.
- **Mais estrelas, mais rápido**
  O nível de procurado controla a frequência: uma estrela pisca devagar, seis é alarme.
- **Simulado antes de soldar**
  Circuito em SPICE e teste no Wokwi; o teste roda o programa contra um `Arduino.h` falso, sem placa.

O JOGO ACENDENDO OS LEDS



CJPHONEHOME é a primeira expressão da Grove que **lê**: ela espera a próxima linha da entrada. No PC é o teclado; na placa, a serial. O mesmo programa serve aos dois.

0,3 s entre a estrela aparecer na tela e o LED acender.

O PROJETO EM NÚMEROS

93
testes passando

2041
linhas de Python

0
dependências

4
saídas: executa, TAC, Python, C++

Dez programas de exemplo, incluindo cinco que só existem para errar: um erro léxico, um sintático, um de digitação com os cheats parecidos, quatro erros semânticos de uma vez e um de execução. Documentação por etapa em `docs/`.

MISSION PASSED

RESPEITO +250.000

Escreva um programa com os cheats e rode no navegador — não precisa instalar nada.

O APP

grove.heybrain.uk

O CÓDIGO

github.com/fernandomcrsilva/grove-language

Gabriel Lacerda · Gabriel Mendonça · Fernando Mendonça — perguntas?